

XtalLoop

AI 实验研发加速器

把研发会议中的实验参数、方案争议、风险判断、待办任务和失败经验，转成可追溯、可审阅、可写回、可复用的结构化研发知识。

Evidence-first

Feishu-native

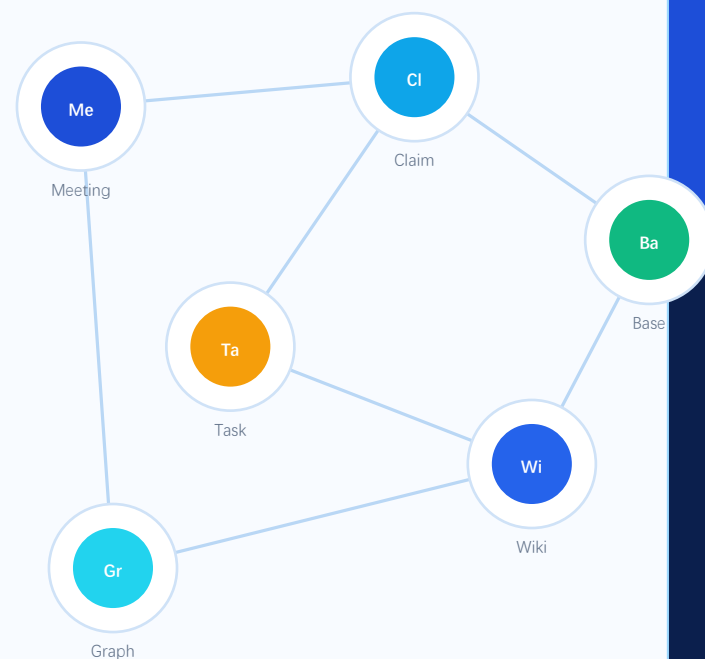
Obsidian Graph

24h 复用

一句话价值

不把 AI 纪要当最终答案，而是把 transcript 拆成带证据锚点的研发 Claim，再进入飞书事实源与团队审阅闭环。

Research Flow Map



真正卡住的不是会议记录，而是研发知识流转

参数依据难追溯

1

温度、转速、配比等调整常在讨论中形成，但会后难定位到说话人、时间戳和原始证据。

方案争议难沉淀

2

争议被压缩成一句结论，后续评审看不到反对理由、风险边界和备选方案。

失败经验难复用

3

高通量实验每天产生大量失败案例，但失败模式没有被索引，导致重复试错。

跨团队节奏不同步

4

算法、实验、物料、研发负责人分布在不同地域，口头共识很快变成信息差。

核心洞察: 会议纪要如果不能变成可审阅的数据对象、可执行的任务对象和可复用的知识对象，就只能减少记录成本，不能减少研发摩擦。

AI 会议助手、ELN/LIMS 与知识库正在融合，但中间缺一层研发语义

01

AI 会议助手

- 强项: 转写、总结、待办识别
- 短板: 参数可信度与来源锚点不足
- 风险: 结论被压缩后失去争议过程

02

ELN / LIMS

- 强项: 实验记录、样品、流程合规
- 短板: 会前/会中决策链路弱
- 风险: 失败讨论留在系统外

03

企业知识库

- 强项: 文档沉淀与权限管理
- 短板: 搜索常停在关键词层面
- 风险: 文档有了，复用仍靠人记忆

XtalLoop 的切入

补齐“会议语义 -> 研发 Claim -> 飞书写回 -> 知识复用”的中间层，让会议 AI 从记录工具升级为实验研发流转基础设施。

以飞书为组织事实源，在上方叠加研发知识结构化 Agent



设计边界: 飞书负责组织级事实源、权限和协作执行; Obsidian 只做个人研究工作台与可重建图谱, 不替代企业事实源。

打通“方案研讨 - 实验执行 - 结果复盘 - 知识沉淀”

”



24 小时内完成的不是“总结一篇文档”，而是六类资产同步落地

实验参数

争议依据

风险提示

待办任务

结果复盘

可复用案例

最小核心不是文档，而是可追溯的 Scientific Claim

Claim Schema

claim_type	parameter_change / risk / task
subject	EXP-DEMO-002
predicate	has_reaction_temperature
object	80°C -> 75°C
verification_status	IN_REVIEW
source.speaker_id	SPEAKER-01
source.start_ms	10000
source.quote_hash	sha256

SourceAnchor 证据锚点

每条 Claim 绑定说话人、时间戳、原文片段与 quote_hash。评审可以回到原始 transcript，而不是只看 AI 摘要。

默认 IN_REVIEW

模型或规则抽取的结论默认不自动发布。关键参数、日期、风险和任务必须经过人工审阅后再写回事实源。

复用但不污染

历史失败案例只作为 reference_id 参与召回，不覆盖当前实验 ID，也不把相似经验误当成当前实验事实。

从会议原文到飞书写回，形成可审计的工程流水线

Transcript Normalizer

统一妙记/纪要/VC 的片段结构，修正说话人、时间戳和会议元数据。

Claim Extractor

抽取参数变化、方案争议、风险提示、待办任务、复盘结论等研发语义。

Evidence Binder

为每条结论绑定 SourceAnchor、quote_hash 与验证状态，支持内容溯源。

Writeback Planner

生成 Base / Task / Docx 写回计划，默认 dry-run，带幂等键。

Review Console

把 IN_REVIEW 结论推入 Base 审阅台，明确责任人与处理状态。

Knowledge Graph

把审阅后的知识沉淀为 Wiki 卡片与 Obsidian 图谱索引。



```
$ npm run demo:e2e
normalized transcript -> extraction bundle
9 IN_REVIEW claims -> 4 writeback commands
simulated dry-run execution log -> report
```

用飞书 CLI / OpenAPI 做可验证接入，公开仓库保留脱敏证据

Reader Profile

- 读取 VC 历史会议、Note、Minutes、transcript。
- 只用于获取当前身份可见的会议产物。
- 不在公开仓库保存真实 token、URL、Open ID、原文。

xtal-reader

Writer Profile

- 写入 Base 审阅表、创建飞书任务、生成 Docx 知识卡片。
- 仅开放白名单命令: record_upsert、task.create、docx.create。
- 真实执行前需要人工确认，默认 dry-run。

xtal-writer

飞书仍是组织事实源，Obsidian 只做个人复用视图

当前仓库已经能跑通 P0 证据链，而不是停留在概念稿



```
$ npm test  
Artifact validation passed.  
Golden-set entries: 24  
Scientific claims: 27  
Meeting transcripts: 2  
Writeback commands: 4
```

飞书能力验证

历史会议查询、会议结束事件、Note 事件、Minutes 事件、transcript 读取、Base 写入、任务创建、Docx 创建均已做脱敏验证。

schemas/

Claim、Bundle、Transcript、Writeback Plan、Execution Log

evaluation/

golden-set、generated bundle、dry-run execution report

fixtures/feishu/

脱敏后的飞书读写与事件验证样例

docs/

PRD、Feishu 可行性、真实业务配置、评委指南

研发场景不能让 AI 直接“想写就写”，必须证据优先、权限收敛

白名单操作

只允许 Base upsert、Task create、Docx create，禁止 raw API、删除、改权限、公开分享。

审阅优先

所有抽取结论默认 IN_REVIEW，高风险参数必须人工确认后进入事实源。

幂等执行

每条写回命令带 idempotency_key，避免重复任务和重复记录。

敏感隔离

真实响应保存在 .tmp 或 private 路径，公开仓库只保留 redacted evidence。

身份拆分

reader / writer profile 分离，降低单一授权过大的风险。

可审计日志

保留 writeback plan 与 execution log，便于复盘每次写入依据。

原则: AI 可以推进整理、比对和计划，但不能绕过证据、审阅和飞书权限边界。

低成本图谱工作台: 让个人研究复用更快, 但事实源仍回到飞书



关键边界: Obsidian 不保存完整真实 transcript, 只同步已授权、已审阅或脱敏摘要; 图谱可重建, 不作为唯一事实源。

把“会后整理”变成可量化的研发流转指标



试点后希望观察的变化



数值为试点目标口径，用于定义评估方式：相同主题重复会议次数、失败案例命中率、任务闭环率、复盘检索耗时。

从可海选 MVP 到企业试点，路径清楚、成本可控



当前状态: P0 工程链路已完成；下一步补齐 3 分钟演示视频与更贴近真实实验的多场景样例。

仓库不暴露真实密钥，但能清楚证明真实业务可落地

1. 看 README

了解真实飞书私有租户如何配置 reader/writer profile、事件订阅、Base/Task/Docx 写回和 Obsidian 边界。

2. 跑本地验证

执行 npm test 与 npm run demo:e2e，检查 schema 校验、claim 抽取、写回计划和 dry-run 日志。

3. 查脱敏证据

fixtures/feishu 中保留读写与事件链路的 redacted JSON，证明不是纯概念方案。

交付印象: 一个 solo 也能完成的低成本、可审计、可扩展研发效能原型；飞书负责组织协同，Agent 负责结构化推进，Obsidian 负责个人知识复用。